Orbit-layout pair table and generator-orbit base

2026-10-22. Study `research/ecbench_generator_orbit_20261022/`. The protocol `PROTOCOL.md` was frozen in commit `d2b3a56c32` before any claim data. Amendment 1 (`de57ca373f`) fixed a `band --json` argument in `run.sh` after card attempt 1 stopped on it; that attempt's outputs were set aside unread. Class: **engineering**.

The sessions ran on `de57ca373f` (`git_dirty: false`):

- `ecbench` binary `f7ac8fb0…`.
- Host class `ECBENV2hebd6c1d47154`, a cloud VM with no CPU reservation. Isolation is L0, so operation counts stand and wall times are descriptive only.
- They ran through the director → mechanical relay. Task card `cards/CARD-ecbench-generator-orbit-20261022-2.yaml` was run by `executor-mechanical`; the receipt is `cards/…-2.receipt.json`.
- `experiment-director` assessed the run as `executed_as_specified`. Its relay check covered the quoted output lines only (`cards/…-2.assessment.yaml`).

Result

The baseline is the lazy orbit-member IC of PR #2158: `compact-orbit-scan` + `mitm-frobenius-lazy-counted` + `stride-renew`. Two changes remove counted work from it.

- **Orbit-layout table** (`ic-layout`; oracle `mitm-frobenius-layout-counted`). It names a hit's Frobenius images by index. It also adds each orbit's own pairs only for rotations $k \le \frac{n-1}{2}$, since $\varepsilon\sigma^{n-k}\big(P + \varepsilon\sigma^{k}P\big) = P + \varepsilon\sigma^{n-k}P$.
- **Generator-orbit base with the generator stride** (`ic-gen-layout`; `generator_orbit=1`, `targets=stride-generator`). Column 0 is the orbit of G, whose logarithm is known. The targets are $Q + [k]G$, with no scalar multiplication and no guard.

Measured (counted `ecbench.gae`, paired by target, cluster-bootstrap 95 %; table 1, figure 1):

- **H1 holds on all 7 curves.** On every one of 960 lazy/layout run pairs, these are identical: trials, relations, rank, recovered logarithm and table entries. Same algorithm.
- **H2: the layout table is an improvement on all 7 curves**, 1.021 × to 1.093 ×, with every lower end above 1. The band order (layout < lazy) holds at $w = 0, 0.1, 1$ on every curve.
- **H3: the generator orbit is an improvement on 6 of 7 curves**, 1.235 × (n37) to 2.94 × (n13, n29). At n17 it is a null: 1.235 [0.980, 1.620]. Its band order (gen-layout < layout) holds at every weight on all 7. The pilot's n23 regression at $w = 1$ (lazy oracle, 4 targets) did not recur.
- **H4: net against the baseline, an improvement on all 7 curves**: 1.26 × to 3.20 ×. The lower end is above 1 everywhere, 1.012 at n17. The band order holds on all 7.

All 5 760 records verified, none is a lower bound, and all 4 320 replays in the 7 audits reproduced. `rho-frob-aa ÷ rho-frob` is 1 [1, 1] on every curve.

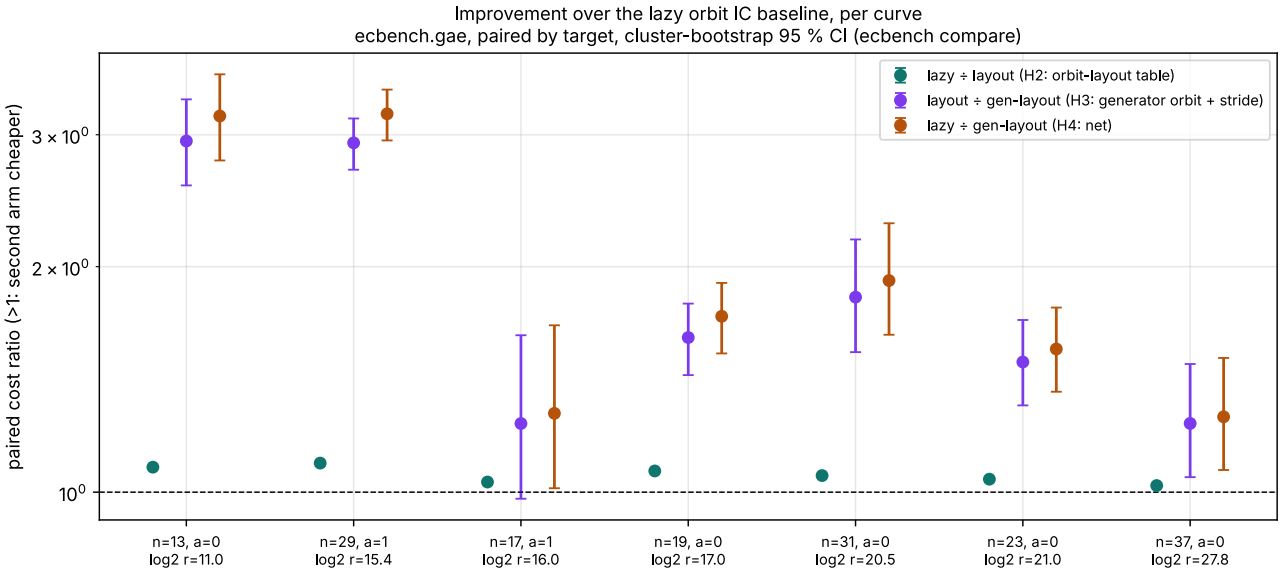


Figure 1: Paired cost ratios per curve against the lazy baseline (H2, H3, H4). Sessions: ECBS1h40b3312cf235 (n13), ECBS1hecda0d013d4e (n17), ECBS1hba7a75763c85 (n19), ECBS1h4f7cdc1ab404 (n23), ECBS1h4217c35be5b6 (n29), ECBS1h573443176608 (n31), ECBS1h28d6e10492db (n37).

curve (a, n, c)	$\log_2 r$	lazy ÷ layout (H2)	layout ÷ gen (H3)	lazy ÷ gen (H4)	gen ÷ rho-frob (H5)
0, 13, 1	11.0	1.080 [1.078, 1.081]	2.943 [2.567, 3.345]	3.178 [2.772, 3.612]	0.739 [0.629, 0.866]
1, 29, 1	15.4	1.093 [1.091, 1.096]	2.926 [2.694, 3.154]	3.200 [2.948, 3.447]	**0.366 [0.241, 0.523]**
1, 17, 1	16.0	1.032 [1.028, 1.036]	1.235 [0.980, 1.620]	1.274 [1.012, 1.670]	1.591 [1.115, 2.189]
0, 19, 2	17.0	1.067 [1.064, 1.070]	1.609 [1.433, 1.785]	1.717 [1.532, 1.903]	1.154 [0.888, 1.468]
0, 31, 2	20.5	1.052 [1.048, 1.057]	1.821 [1.538, 2.175]	1.916 [1.622, 2.285]	1.154 [0.878, 1.502]
0, 23, 3	21.0	1.041 [1.038, 1.044]	1.492 [1.306, 1.698]	1.553 [1.362, 1.764]	1.934 [1.414, 2.475]
0, 37, 8	27.8	1.021 [1.018, 1.023]	1.235 [1.047, 1.483]	1.261 [1.071, 1.511]	3.886 [3.165, 4.790]

Table 1: Paired ratios at calibration prices, cluster-bootstrap 95 % intervals (`ecbench compare`; `sessions/*/comparisons/`). 96 pairs per curve (32 targets × 3 rounds); 192 at n29, 48 at n37. "gen" is `ic-gen-layout`. In the last column, below 1 means the IC arm is cheaper than rho.

Distance to rho and the §5a band (H5)

curve	gen-layout S at w = 0 / 0.1 / 1	lazy S at w = 0 / 1	rho-frob S	`ecbench band --ranks`: gen vs rho-frob
n13	1.406 / 1.536 / 2.706	4.540 / 7.190	1.987	unresolved inside the band
n29	**0.546 / 0.611 / 1.196**	1.780 / 3.323	1.569	**gen-layout < rho-frob**
n17	1.845 / 2.142 / 4.810	2.387 / 6.105	1.240	rho-frob < gen-layout
n19 [a]	1.081 / 1.081 / 1.081	1.856 / 1.856	0.937	rho-frob < gen-layout
n31	0.559 / 0.632 / 1.286	1.079 / 2.674	0.512	rho-frob < gen-layout
n23	0.838 / 0.945 / 1.908	1.304 / 3.438	0.443	rho-frob < gen-layout
n37	0.772 / 0.892 / 1.965	0.972 / 3.059	0.211	rho-frob < gen-layout

Table 2: Mean S per arm under the §5a band (`tables/band-*.json`). The band charges every calibration-priced and unpriced native operation at w GAE. rho-frob's natives are priced by `full_prices.json` and are not re-priced, so its S does not depend on w. [a] n19 has no pinned calibration row; its IC natives carry live prices that the band does not re-price, so the band is not shown there (see Limits).

n29: IC is ranked ahead of the matched rho. `ic-gen-layout ÷ rho-frob` is 0.366 [0.241, 0.523] over 192 pairs. The IC arm's band S (0.546 / 0.611 / 1.196) is below rho-frob's 1.569 at all three weights, and `ecbench band --ranks` returns `ic-gen-layout-c1 < rho-frob`. By the protocol's H5 rule, IC is ahead of rho on this curve.

How robust the ranking is to the band's definition. The independent validator (finding F3) showed that `ecbench band` is not applied alike to every arm. It charges the IC arms' calibration-priced natives at w, but leaves rho's natives (its canonicalisations and distinguished points) at `full_prices.json` rates. Table 3 adds two bands that treat both arms alike.

- The **like-for-like** band (`analysis/core_band.jq`) charges the same operation types on both arms at w. The ranking holds there.
- The validator's **every-unit-operation** band charges each field squaring, hash probe, point negation and the factor-base byte floor at w. Only the IC records count work at that granularity. In that band the IC arm loses at $w = 1$.

So the n29 ranking holds under the protocol's instrument and under the like-for-like band. It is unresolved under the strictest accounting.

band	ic-gen-layout S (w = 0 / 0.1 / 1)	rho-frob S	order
`ecbench band` (protocol)	0.546 / 0.611 / 1.196	1.569 / 1.569 / 1.569	IC ahead at every weight
like-for-like	0.413 / 0.478 / 1.063	1.503 / 1.582 / 2.288	IC ahead at every weight
every unit operation (validator)	0.413 / 0.887 / 5.145	1.503 / 1.582 / 2.288	unresolved

Table 3: n29 under three §5a bands. The like-for-like band at the other curves: n13 IC ahead at every weight (0.892 / 1.022 / 2.192 against 1.911 / 1.971 / 2.505), which the protocol's instrument leaves unresolved; n19 unresolved; rho-frob ahead at n17, n23, n31 and n37 (`analysis/core-band-*.json`).

What it is, and what it is not. At $c = 1$ the pipeline is a Frobenius-folded baby-step giant-step (protocol §2).

- The baby steps are the 26 pair-sum classes of G's signed orbit, covering 1 508 of $r = 42457$ points.

- The giant steps are $Q + [k]G$.
- The arm averages 35.0 trials, against the 33.7 that the coverage's gaps predict from a uniform start.

Its 118 GAE beats rho-frob's 323 GAE (figure 3) because rho-frob spends a fixed 133 GAE on set-up at this size. rho-frob without its set-up (search 157, natives 13, verification 20: 190 GAE; its S is heavy-tailed, median 0.98) would beat the IC arm's 246 GAE at $w = 1$.

So this is the first measured, band-robust crossing of the matched rho by an IC-pipeline arm on a curve in this repository. It is a small-r effect of a generic mechanism, and it is not "faster than rho": at n31 the IC arm is 1.15× rho-frob, at n23 1.93× and at n37 3.89×, and the gap grows with r (figure 2). n13 crosses at $w = 0$ and 0.1 but not at 1, so it is unresolved.

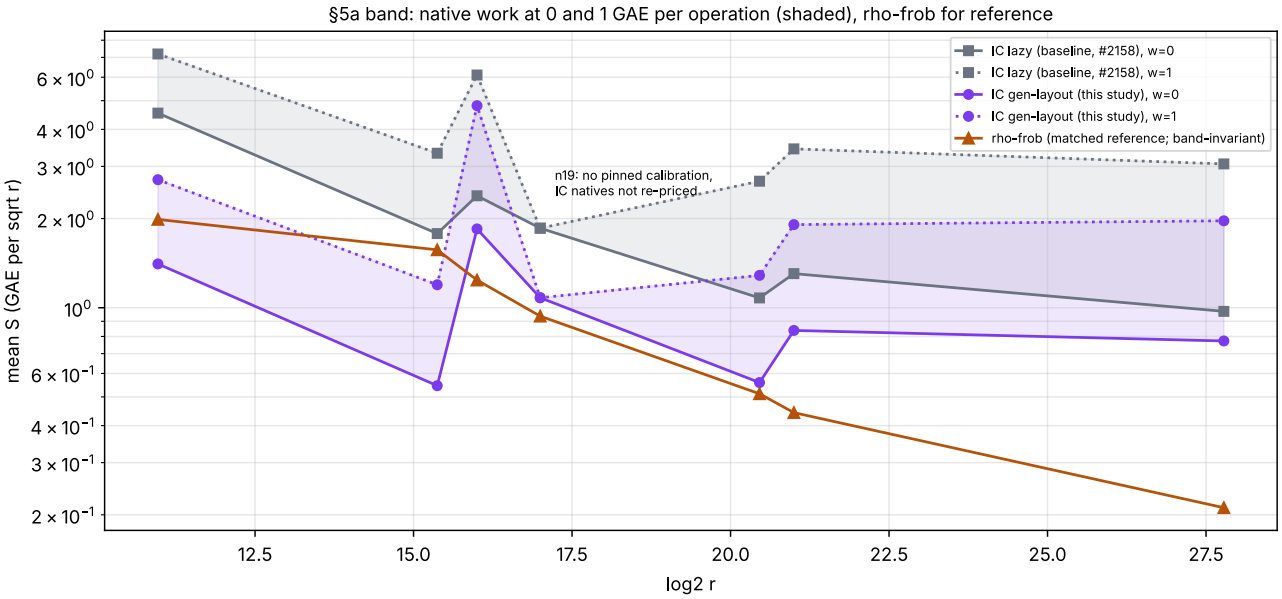


Figure 2: Mean S at the band's ends ($w = 0$ solid, $w = 1$ dotted, range shaded) for the lazy baseline and the gen-layout arm, with rho-frob, against $\log_2 r$.

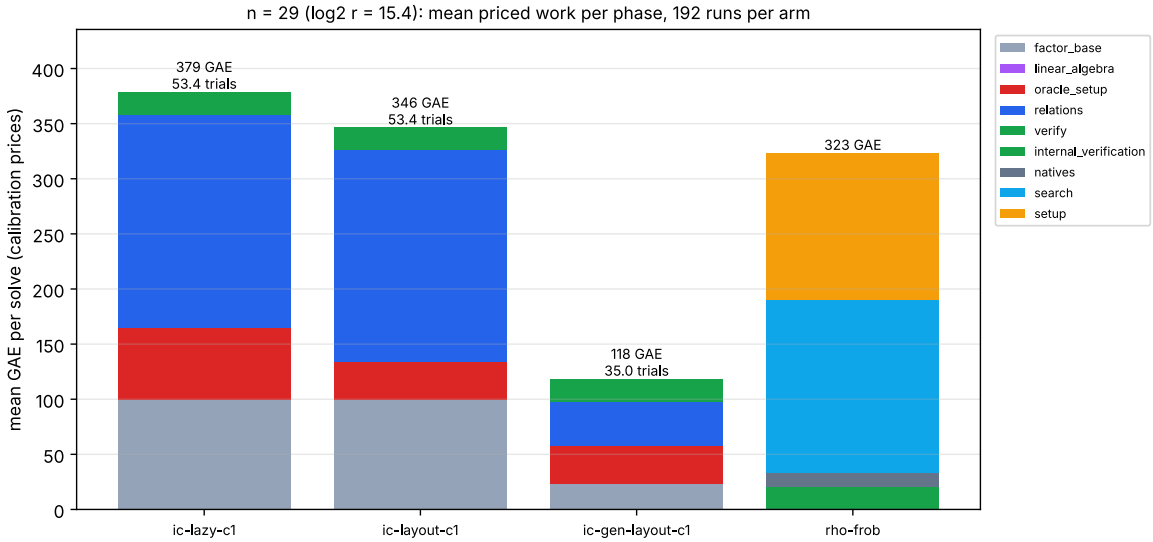


Figure 3: n29: mean priced work per phase, calibration prices (derived: means over 192 non-warm-up records per arm, `figures/summary.json`). The generator orbit removes the scanned seed's lift, projection and order check (factor_base 100 ⟶ 23 GAE). It also removes the relation phase's scalar multiplications. rho-frob's set-up is 133 of its 323 GAE.

Validation

An independent validator agent did the following (`validation/`):

- It replayed every session (`verify --replay-all`): 4 320 of 4 320 replays reproduced.

- It verified all 5 760 scalars twice, once through the audit and once with its own GF(2^n) code that shares nothing with the repository.
- From the raw `records.jsonl` it recomputed all 84 band means, H1 (960 of 960 pairs equal) and the n29 trial mean. Every one agrees exactly.

`VALIDATION.json` names the replay certificate, the validator report and the rho-frob reference, with the n29 band and all three bands above. `scripts/validation_gate.py evidence` reports the study directory `valid`.

The validator also found four problems. None changes a verdict here:

- **F2.** `band.rs` subtracts the calibration-priced C from a G that `*_full` re-priced at `full_prices.json` rates. Correcting this lowers the IC means by 1.2–12.3 %.
- **F3.** The band is not applied alike to every arm (table 3).
- **F4.** n19 is unpinned (below).
- **F5.** The A/A control uses the same seeds, so it tests determinism only and cannot fail.

F2 and F3 are band-tool follow-ups.

Classification and limits

Engineering (AGENTS.md success criterion). The layout table performs less native work for the same relations. The generator orbit replaces one scanned column with a known one and draws targets without scalar multiplication. No exponent changes. This is neither relabelling nor accounting: H1 shows the layout arm finds the same relations, and every saving is work no longer performed, charged at the same prices.

n19. `docs/ic/calibration.json` has no pinned row for this curve. Its IC natives (canonicalisations and lookups) are priced by a live calibration, and `band.rs` neither strips nor re-prices them. The band there is the calibration-price cost at every weight. A band-tool gap, already present for every unpinned curve, and a follow-up.

Other limits.

- Wall times are not admitted (L0).
- Prices are one host's microbenchmark ratios.
- The replay certificate comes from an independent validator on the same host class; operation counts do not depend on the host. It is not the cross-host replay that `ecbench claim build` requires.
- Per-curve columns are those of the 2026-10-14 protocol, not retuned.

Nothing here concerns larger fields, ECC2K-130, deployed curves or security.

Evidence:

- `sessions/`: 7 ecbench sessions, loose and CI re-auditable.
- `audits/`, `tables/` (`table-`, `band-` and `compare-` files) and `logs/`.
- `cards/`: both task cards, receipts and the director's assessment.
- `figures/summary.json`, from `figures/make_summary.sh` (jq), and `figures/plot_figures.py`, which lays out recorded values only.
- `validation/`: replay certificate, validator report and findings. `VALIDATION.json`.
- `analysis/core_band.jq` and `analysis/core-band-*.json`: the like-for-like band.
- Unit tests `layout_table_returns_the_lazy_pairs_with_no_maps_and_half_the_same_orbit_sums` and `generator_orbit_base_pins_the_target_with_one_relation`. `cargo test --release --lib -- ic_framework ic_boundary ecbench`: 221 passed.